City Network & Job Scheduler

Design and Analysis of Algorithms - Assignment 3. Yelnur Pashanov, SE2523. Release v1.0.

1. Design and complexity

One adjacency-list Graph stores int edge weights, directed/undirected mode, logical input edges and cached in-degrees. Algorithms use arrays, ArrayList, ArrayDeque and PriorityQueue; no graph library or design-pattern framework is used. Totals and distances use long to avoid int overflow. MST returns a forest on disconnected input. SCC uses explicit stacks, and Kahn rejects cycles.

Operation	Time	Extra space	Reason
DSU makeSet	O(V)	O(V)	Initialize parent and size arrays.
DSU find	O(alpha(V)) amortized	O(1) per call	Iterative full path compression; shared DSU storage is O(V).
DSU union	O(alpha(V)) amortized	O(1) per call	Two finds; attach the smaller tree below the larger root.
Kruskal	O(V + E log(E+1) + E alpha(V))	O(V + E)	Copy and sort E edges, then test edges with DSU.
Prim (lazy heap)	O(V + E log(E+1))	O(V + E)	Scan adjacency entries and insert/remove at most O(E) queue items.
Kosaraju	O(V + E)	O(V + E)	Two iterative DFS passes plus a reversed graph.
Kahn	O(V + E)	O(V)	Each vertex leaves the queue once; every DAG edge reduces one in-degree.
DAG shortest paths	O(V + E)	O(V)	Kahn order, source initialization and one relaxation pass.

Extra space excludes the input graph and includes returned arrays/lists. Lazy Prim can store parallel edges, so log E is the precise bound here; for a simple connected graph E <= V(V-1)/2 and this is the usual O(E log V). With both DSU heuristics, alpha is the inverse Ackermann function, which grows extremely slowly. This is an amortized sequence bound, not a worst-case constant-time promise for each find.

Counter definitions and input control

dsu_ops counts find invocations plus successful unions; it excludes parent-pointer steps. edge_scans counts Kruskal's post-sort candidates, Prim's adjacency entries, the two Kosaraju DFS passes, Kahn's queue-processing edges, or DAG relaxations. Graph loading, reversal construction and validation are outside the scan counters. DAG timing includes Kahn preprocessing; its own scan counter covers only relaxation. comparisons counts weight-comparator calls in sorting/heap operations or candidate-distance comparisons in DAG relaxation. max_depth is the maximum occupied explicit DFS stack; zero means no DFS stack was used.

The official benchmark has fixed sizes and densities, rather than a student-selected input count. Interactive manual input accepts arbitrary n and m, generate writes a custom graph, and benchmark-custom accepts user-selected size/density lists. These extra modes do not replace the required 50-row run.

2. Correctness and algorithm choice

Cut property: Kruskal and Prim

Consider a cut that respects the forest already chosen. Let e be a minimum-weight edge crossing this cut, and take an MST T containing the chosen forest. If T omits e, adding e creates a cycle containing another crossing edge f. Since weight(e) <= weight(f), replacing f with e preserves connectivity and cannot increase the total weight. Kruskal applies this exchange argument to the DSU components, while Prim applies it to visited versus unvisited vertices. Repeating safe additions yields an MST in each connected component, so both methods return minimum spanning forests even when the input is disconnected.

Induction on the topological order

Every edge of a DAG goes from an earlier vertex to a later vertex in a valid topological order. Initialize exactly the zero-in-degree vertices to distance zero, as if a virtual super-source had zero-weight edges to them. For the induction step, assume that all predecessors of the current vertex already have their optimal distances. Each predecessor has relaxed its outgoing edge, so every possible final edge of a path to the current vertex has already been considered. The current distance is therefore optimal and its outgoing edges can be relaxed once, recording a parent whenever a distance improves. Negative weights are safe because a DAG has no cycle and no path can return to an already processed vertex; in a general graph negative edges invalidate Dijkstra's greedy rule, and a reachable negative cycle can make a finite shortest distance undefined.

Situation	Choice and reason
Sparse undirected graph / edge list	Kruskal: sort the available edge list and use DSU for cheap cycle rejection.
Adjacency-list network / grow from a district	Prim: frontier edges are natural; compare heap work with sorting before choosing on dense input.
Very dense simple graph / matrix already given	A matrix Prim variant can use O(V^2); this project implements the required PriorityQueue variant.
Directed graph may contain cycles / deadlocks	Kosaraju: find mutually reachable groups; a multi-vertex SCC or a singleton self-loop signals a cycle.
Need only a job order or cycle detection	Kahn: queue zero-in-degree jobs and reject an incomplete order.
DAG routing, possibly negative weights	Topological shortest paths: O(V+E), including all source jobs. General negative graphs need a method such as Bellman-Ford.

Validation

Tests	Independent evidence
DSU	1,000 random unions vs simple labels; component counts and exact counter calls.
MST	100 connected random graphs: equal weights and V-1 edges; 25 tiny cases vs exhaustive spanning trees.
SCC	50 random graphs (one with V=200): compare mutual BFS reachability; verify exactly 2E DFS scans.
Topology / paths	Cycles rejected; order validates every edge; 100 random negative-weight DAGs vs Bellman-Ford; parent paths checked.
Safety / inputs / benchmark	100,000-vertex path reaches explicit depth 100,000; four input files, edge cases, reference and custom-run tests. All 17 JUnit tests pass.

3. Reproducible measurements

Each graph kind starts with a fresh java.util.Random(42). Shuffle order, tree-edge draws and weight draws follow the specification; when b equals a, only b is redrawn. The ten (n,m) pairs create 30 graphs and 50 algorithm/graph rows. Each algorithm runs five times on the same graph; the CSV stores the median time and internally counted, repeat-checked counters. Generation and result validation are outside timed sections. All ten reference triples match exactly, and Kruskal/Prim agree automatically on every run set.

n	m	MST weight	SCCs	DAG distance sum
1,000	4,000	147,970	27	396,274
1,000	20,000	30,363	1	252,411
3,162	12,648	468,911	137	1,271,911
3,162	63,240	97,589	1	723,157
10,000	40,000	1,486,139	373	3,941,828
10,000	200,000	309,550	1	2,381,342
31,623	126,492	4,715,515	1188	12,551,891
31,623	632,460	968,517	1	7,376,612
100,000	400,000	15,012,094	3923	40,096,539
100,000	2,000,000	3,056,681	1	23,512,087

Counters divided by complexity formulas

Algorithm	Normalization	Sparse range	Dense range
Kruskal	comparisons / (E log2 E)	0.745-0.907	0.666-0.903
Prim	comparisons / (E log2 E)	1.829-1.934	1.825-1.890
Kosaraju	scans / (V+E)	1.600-1.600	1.905-1.905
Kahn	scans / (V+E)	0.800-0.800	0.952-0.952
DagShortestPath	scans / (V+E)	0.800-0.800	0.952-0.952
DSU inside Kruskal	dsu_ops / scanned edge	2.134-2.324	both densities

Traversal ratios are exactly constant at each density. Sort/heap ratios remain of comparable size rather than being numerically identical; finite samples, repeated weights (only 1..1000) and adaptive Java sorting affect them. For Kruskal, dsu_ops = 2*scans + accepted_edges, so the call ratio above follows from counted operations. normalized.csv also records dsu_ops/E, which varies with early stopping. Call counters cannot measure alpha(V) itself, and a finite plot supports rather than proves an asymptotic bound.

Wall-clock time

Measured medians range from 0.456 to 1757.603 ms on Java 17.0.20.1, Windows 11 amd64, with a 2 GiB heap. At V=100,000 and E=2,000,000, Kruskal took 399.676 ms and lazy Prim 1757.603 ms. There is no explicit warmup; JIT compilation, garbage collection and background load especially affect small cases. Different hardware and JVMs will change timings, so these observations are secondary to the operation counters. The runtime details are saved beside the CSV.

4. Operation growth and scans per edge

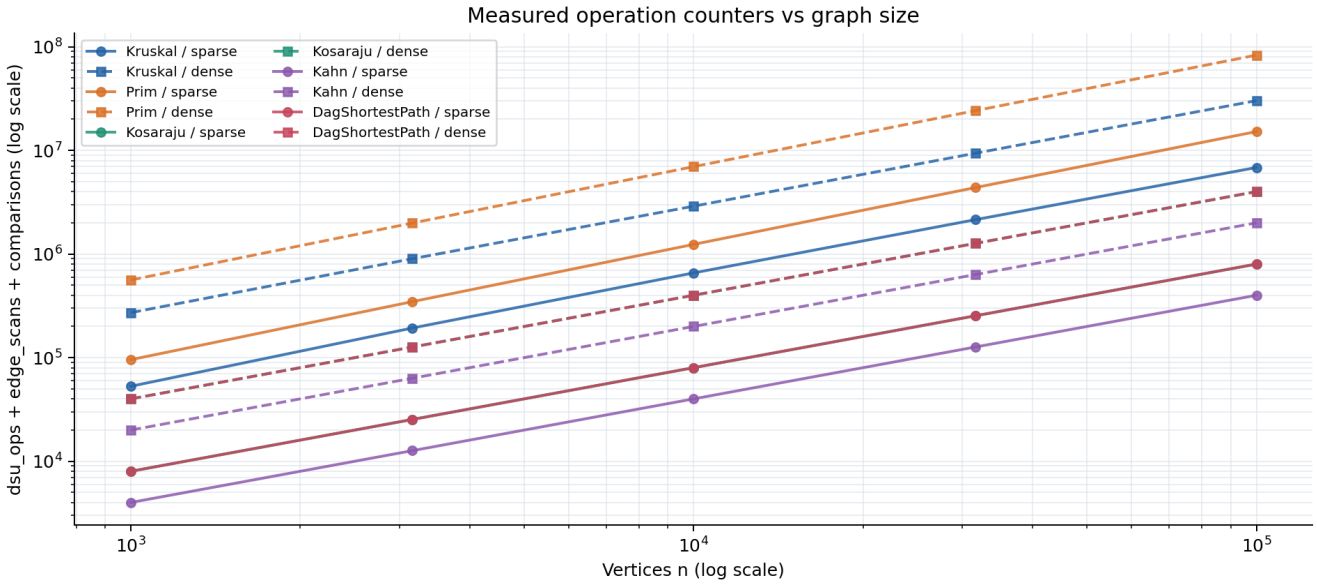


Figure 1. Log-log operation growth; solid lines are sparse and dashed lines are dense. Operations are the sum of three different counters, an illustrative work proxy rather than a time model. Kosaraju and DAG relaxation overlap because each contributes 2E to this sum.

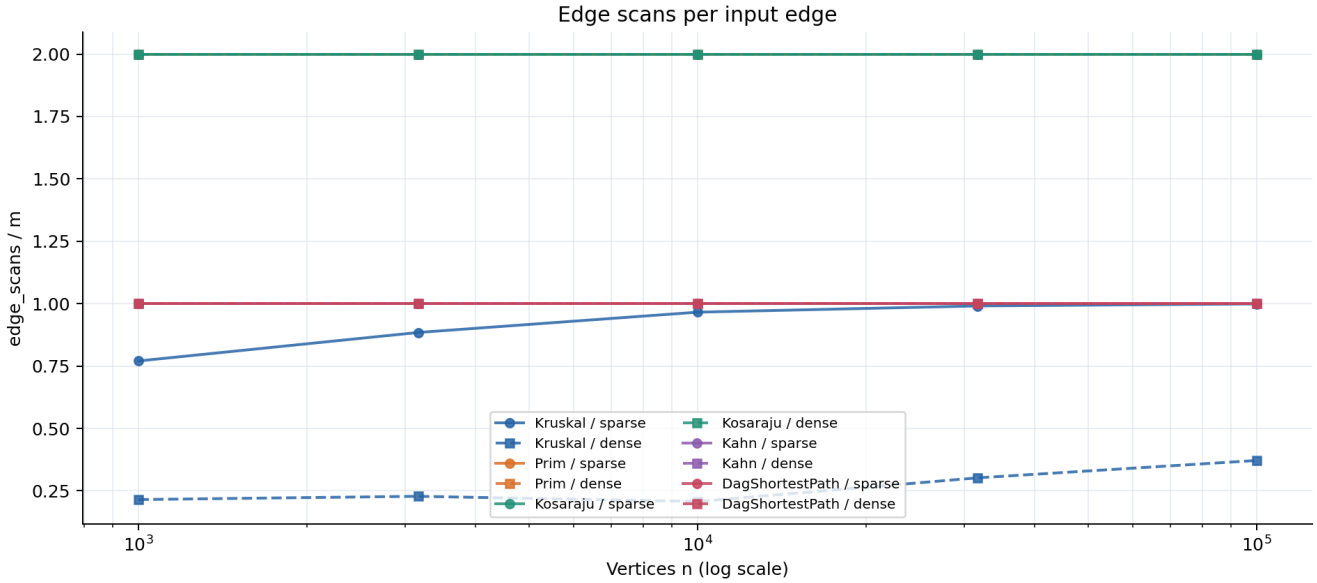


Figure 2. Prim and Kosaraju scan 2E entries, while Kahn and DAG relaxation scan E on these DAGs. Overlapping lines have the same measured values. Kruskal's fraction changes because it stops after V-1 accepted edges, although it sorts the complete edge list first.

5. Kruskal's edge share and fitted slopes

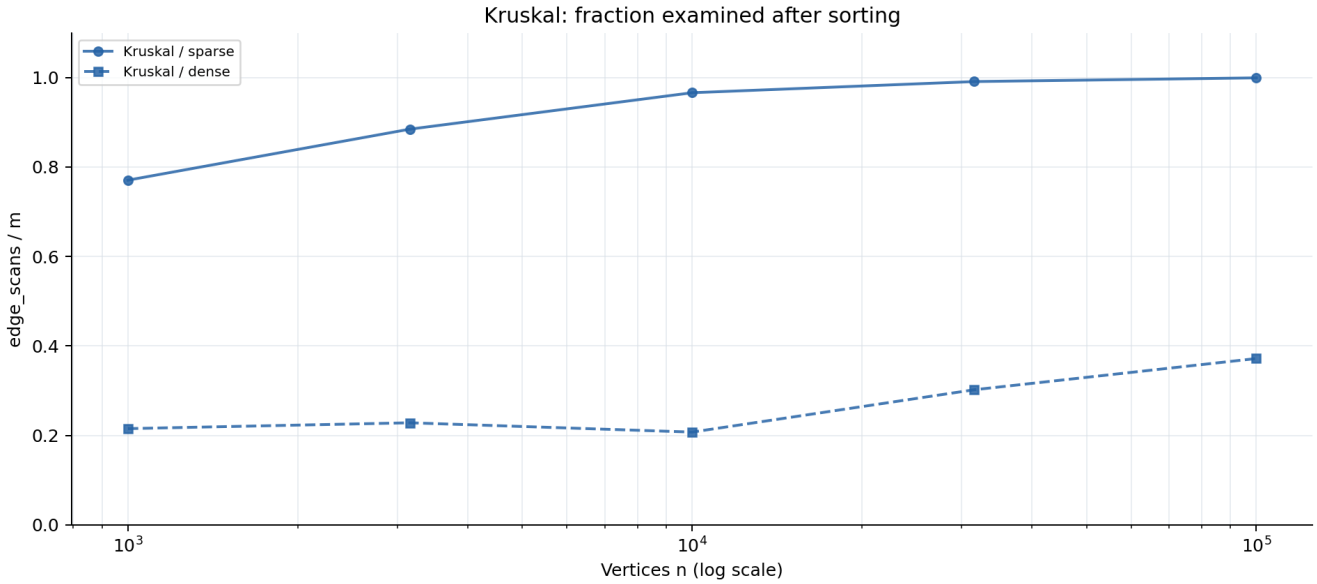


Figure 3. Kruskal examines about 77.1%-99.9% of sparse edges and 20.7%-37.2% of dense edges after sorting. The saving is in DSU candidate processing, not in sorting. Lazy Prim still scans both adjacency directions and performs heap operations; density alone does not determine a winner.

Algorithm	Scan slope sparse	Scan slope dense	Comparison slopes sparse / dense
Kruskal	1.055	1.119	1.055 / 1.016
Prim	1.000	1.000	1.108 / 1.090
Kosaraju	1.000	1.000	not applicable
Kahn	1.000	1.000	not applicable
DagShortestPath	1.000	1.000	not applicable

Slopes are least-squares fits of ln(counter) against ln(n), separately at each density, using all five sizes. Linear traversals give 1.000; Kruskal's scan slopes are 1.055 and 1.119. The specification attributes the latter to sorting, but post-sort edge_scans do not count sort comparisons: early stopping explains the changing scan fraction. Sorting's worst-case E log E factor belongs to comparisons. The measured comparison slopes and normalized ranges are reported as observed, including the effect of a fixed 1,000-value weight domain.

Specification notes and reproduction

The deliverables section's phrase 'all four generated graphs' conflicts with the detailed ten-pair benchmark and 50-row requirement; this project follows the detailed specification. The Lecture 6 Kruskal pseudocode has an equality typo: an edge is accepted only when its endpoints are in different DSU components. Optional Tarjan/condensation bonuses are outside this core submission. After installing the documented prerequisites, python tools/reproduce.py rebuilds, tests, benchmarks and regenerates these plots and both report formats.

Sources: DAA_Assignment3_Graphs.pdf, sections 1-10 and rubric; Lecture06 - Disjoint Sets and MST.pdf, slides 3-13. Lecture 5 - Hashing and Trees was reviewed but is not the algorithmic basis for this assignment. Repository: https://github.com/aokoshi/DAA-Assignment3 (main, tag v1.0). AI acknowledgement: OpenAI Codex assisted with the implementation, tests, measurements and report; the student must review and understand the submitted work.